

QUOTA — Bounty Validation

Test applied to every bounty: **does QUOTA break or get materially worse if this sponsor's component is removed?** If not, it's decoration and we drop it. "Meets the minimum rules" is not enough.

Verified = read in the sponsor's docs this session. Unverified items are gated in Phase 0 (see `03-phases.md`).

Verdicts

#	Bounty	Prize	Verdict	Role in QUOTA	Confidence
0	Track 4: Trust, Identity & AI Infra	\$10k ×3	Core	The whole project	High
6	Privy	\$5k	Core	Agent-side wallet infrastructure + policy engine + secret derivation	High
1	Dynamic	\$5k	Core	Service-side wallets: autonomous slasher + reward custody via delegated access	Medium
5	Alibaba Qwen 3.8 Max	\$5k credits	Core (demo workload)	The autonomous agent that spends quota; article required	Medium
4	Nansen	\$5k pool	Conditional → Phase 6	Powers the opt-in "Screened" tree (sybil/recidivism screening of stakers)	Low–Medium
3	Cleanverse CVI/CVA	\$2k	Conditional → Phase 0 gate	"Compliant" tree: stake held as CVA, moves only between CVI-verified wallets	Unknown (docs gated)
2	Monad Foundation: Community Team	\$5k	Eligibility only	None, it's a profile setting	Depends on you

Why each verdict

Privy — Core. "Beyond login" is satisfied by features QUOTA genuinely needs:

- Agents are autonomous software. Each needs a wallet no human is holding. A **Privy server wallet** with the **operator's user as owner** and our runtime **authorization key as additional signer** gives exactly "agent acts alone, human can still change the rules" (verified: owners, additional signers, policy overrides, key quorums, user-passkey ownership are all documented).
- **Policy engine** restricts the agent wallet to QuotaRegistry calls and a value cap, so a hijacked agent runtime can't drain funds.
- **Secret derivation:** the RLN secret `a0` is derived from a wallet signature over a fixed domain string, so it is recoverable with the wallet and never stored. (Assumes signatures are deterministic. Phase 0 test; fallback = encrypted blob.)
- **Sign-only mode:** we sign with Privy but broadcast ourselves, because BTX submission needs our own path.
- Multiple features used → qualifies for the "multiple Privy features" bonus.
- Subsidized testnet usage: Monad docs say email `monad@privy.io` .

Dynamic — Core, but the riskiest "core." QUOTA has two real actors, and each needs a wallet:

- *Agent operator* → Privy (above).
- *Service operator* (runs the MCP server/API) → **Dynamic**: embedded wallet + auth for the service console, **delegated access** so the backend slasher can sign `slash()` from the operator's wallet without a prompt, rewards land in the operator's own wallet. Server wallets as the fallback if delegated access is awkward on Monad.
- This is a genuine split, and it lets us show QUOTA is wallet-agnostic ("agent on Privy, service on Dynamic, same protocol"), which is also a developer-experience point.
- **Risk**: Dynamic's docs don't describe spend policies for these wallet types, and Monad is an EVM custom network setup to confirm. Both are Phase 0 spikes. **Cut rule**: if Dynamic isn't signing a Monad tx by end of Phase 0, drop it. We do not fake it.

Qwen — Core for the demo, not for the protocol. Honest framing: the protocol doesn't need an LLM. But QUOTA exists *for* agent traffic, so the demo needs a real agent whose behavior depends on quota:

- **"Scout"**, a Qwen-driven agent with tools `quota_status`, `call_tool(server, tool, args)` (SDK attaches the ZK proof), `topup_stake`, `switch_server`. It plans multi-step research across several QUOTA-gated MCP servers, treats quota as a budget, backs off before the limit, and decides when raising its tier is worth it. That is real planning + tool use + on-chain decision-making.
- A **violation mode** (a deliberately misbehaving agent) drives the slash demo.
- The article (required) writes itself from the build log.
- Verify at signup: the exact model ID / endpoint on qwencloud.com. Keep the agent loop provider-agnostic so a model swap is one line.

Nansen — Conditional, and the first thing cut. The rules are easy to meet (verified: Monad is supported, incl. Smart Money / Token God Mode / Profiler, and x402 on Monad). The hard part is a *crucial* role. A dashboard that shows wallet charts would be decoration. The honest design:

- QUOTA's sybil story is the stake, but there is one gap: **recidivism and stake-concentration**. A slashed operator can re-stake from a related wallet.
- A **Screenener** (permissionless; anyone can run one) uses Nansen **Profiler** endpoints (labels, related wallets, funding source, counterparties) on a new operator wallet at enrollment, and if clean, signs an attestation. Attested leaves go in a separate **Screened tree**. Services choose which trees they accept: Open, Screened, or both.
- Without Nansen, the Screened tier doesn't exist. Screening is opt-in and pluggable, so it does not make Nansen a gatekeeper (the track's anti-capture test).
- Also feeds the service console: stake concentration, funding clusters, slash history per tree.
- **Cut rule**: if Phase 5 overruns, drop Phase 6 Nansen first. Needs an API key (docs host failed a cert check from here; confirm key/credits flow in Phase 0).

Cleanverse — Conditional, decided by a Phase 0 gate. The criteria demand identity verification *structurally coupled* to asset movement. Possible honest coupling:

- A **Compliant tree** where the stake is a **CVA token**. Staking and slash payouts move CVA, which only transfers between **CVI-verified** wallets. A regulated data seller (e.g., a financial data API) can only be paid penalties by verified operators.
- Anonymity is preserved where it matters: requests stay unlinkable. Only the *operator* (who is public at enrollment anyway) is verified.
- Tension with the thesis: CVI is a centralized issuer, which is exactly what QUOTA's open tree avoids. Resolve by making Compliant strictly opt-in per service, never default, and say so in the pitch.

- **Blocker:** docs are behind an invitation code and the guides are Google Drive links, so I could not read them. Phase 0 gate: confirm (a) a CVA exists on Monad testnet, (b) CVI check is callable on-chain from our contract, (c) sandbox credentials work. Fail any one → drop it (\$2k isn't worth the risk).
- The pasted App ID and API key belong in `.env` only. They are not copied into these docs or the repo.

Monad Foundation Community Team — Eligibility only. No product role needed. It runs alongside everything else. Requirement: your team's profile must name an **onboarded campus/community group**. I can't tell whether that applies to you. Tell me the community, or skip it.

What this means for scope

Layer	Always ships	Ships if gates pass
Protocol	Registry, passkey custody, RLN proofs, BTX slash	Screened tree (Nansen), Compliant tree (Cleanverse)
Wallets	Privy agent wallets	Dynamic service wallets
Demo	Qwen "Scout" agent, MCP server behind QUOTA, console	—

Prize exposure if everything lands: Track 4 + Privy + Dynamic + Qwen + Nansen + Cleanverse (+ Community). If the gates fail we still have Track 4 + Privy + Qwen with a clean, honest build.